

# 1 Problem 1: Consider the lung cancer data and solve the following questions using Python:

## 1.1 Basic EDA Questions

- i) Summarize the distribution of Age. Is it normally distributed or skewed?
  - ii) What proportion of individuals are smokers vs non-smokers?
  - iii) Which income group has the highest frequency?
  - iv) What percentage of individuals are exposed to high pollution?
  - v) What is the overall prevalence of lung disease?
- 

## 1.2 Association & Crosstab

- i) Is there an association between Smoking and Lung Disease?
  - ii) Interpret using a contingency table.
- 

## 1.3 Does Pollution Level Affect Lung Disease?

- i) Compare lung disease prevalence across Income groups.
  - ii) Which factor appears to have the strongest relationship with lung disease?
- 

## 1.4 Statistical Testing

- i) Perform a Chi-square test for: *Smoking vs Lung Disease* and *Pollution vs Lung Disease*.
  - ii) When should you use Fisher's Exact Test instead of Chi-square?
  - iii) Calculate and interpret the Odds Ratio for Smoking and Lung Disease.
- 

## 1.5 Correlation & Interpretation

- i) Interpret the correlation between:
    - Smoking and Lung Disease
    - Age and Lung Disease
  - ii) Why is correlation not sufficient for causal inference?
- 

## 1.6 Logistic Regression

- i) Fit a logistic regression model:

$$\text{LungDisease} \sim \text{Smoking} + \text{Age} + \text{Pollution} + \text{Income}$$

- ii) Which variables are statistically significant predictors?
- iii) Interpret the odds ratio of Smoking.
- iv) What happens to the effect of Smoking after adjusting for Pollution?

---

## 1.7 Advanced Modeling

- i) Add an interaction term:

Smoking  $\times$  Pollution

- ii) Is the interaction significant?
  - iii) Interpret the interaction.
  - iv) Does pollution amplify smoking risk?
- 

## 1.8 Model Evaluation

- i) Compute the ROC curve and AUC.
  - ii) Is the model good?
  - iii) What is the confusion matrix?
  - iv) Discuss model accuracy vs sensitivity vs specificity.
- 

## 1.9 Research Question

“Using the lung disease dataset, perform exploratory data analysis and fit a logistic regression model to identify key determinants of lung disease. Interpret your findings and discuss public health implications.”

## 1.10 Basic EDA Solutions

- i) Summarize the distribution of Age. Is it normally distributed or skewed?

```
[5]: import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import statsmodels.api as sm
import statsmodels.formula.api as smf
from scipy.stats import chi2_contingency
from sklearn.metrics import roc_curve, auc, confusion_matrix, \
    classification_report, accuracy_score

# 1. Load Data
# folder path (NOT file)
os.chdir('E:/python')

# file read
df = pd.read_csv('lung_disease.csv')
```

```

# Pre-processing: Convert categorical variables to binary/numeric for modeling
df['Smoking_binary'] = df['Smoking'].map({'Yes': 1, 'No': 0})
df['LungDisease_binary'] = df['LungDisease'].map({'Yes': 1, 'No': 0})
df['Pollution_binary'] = df['Pollution'].map({'High': 1, 'Low': 0})
# i) Age Distribution
# Summary statistics

print(df['Age'].describe())
sns.histplot(
    df['Age'],
    kde=True,
    bins=10
)

plt.title("Distribution of Age")

plt.xlabel("Age")
plt.ylabel("Frequency")

plt.show()
#Skewness
print(
    "Skewness:",
    df['Age'].skew()
)

#Normality Test
#Using the Shapiro-Wilk Test

from scipy.stats import shapiro

stat, p = shapiro(df['Age'])

print("Shapiro Test Statistic:", stat)
print("P-value:", p)

if p > 0.05:
    print("Age is approximately normally distributed")
else:
    print("Age is not normally distributed")

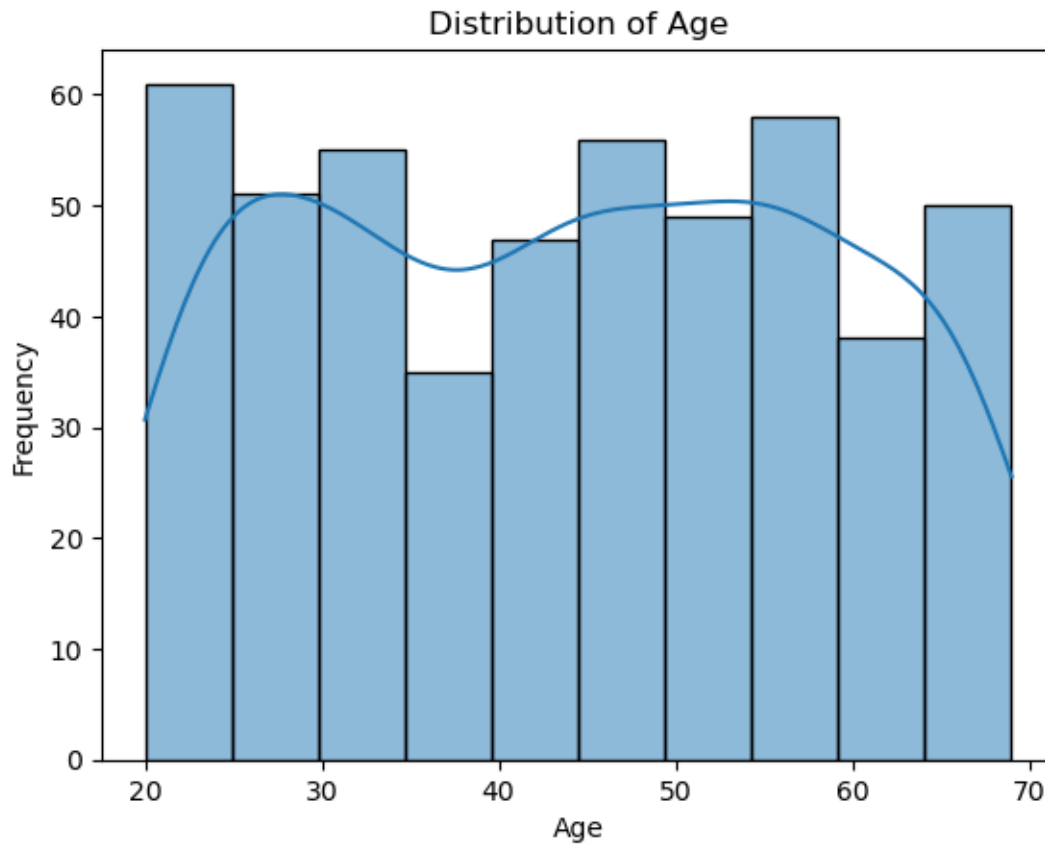
```

```

count    500.000000
mean      43.904000
std       14.604841
min       20.000000
25%       30.750000

```

```
50%      45.000000
75%      56.000000
max       69.000000
Name: Age, dtype: float64
```



```
Skewness: 0.012141455267137275
Shapiro Test Statistic: 0.9488992946952997
P-value: 4.035055085974128e-12
Age is not normally distributed
```

ii) What proportion of individuals are smokers vs non-smokers?

```
[6]: #ii
      # Count smokers and non-smokers

      smoking_counts = df['Smoking'].value_counts()

      print(smoking_counts)

      # Percentage of smokers and non-smokers
```

```

smoking_percentage = (
    df['Smoking'].value_counts(normalize=True)
) * 100

print(smoking_percentage)

```

```

Smoking
No      293
Yes     207
Name: count, dtype: int64
Smoking
No      58.6
Yes     41.4
Name: proportion, dtype: float64

```

iii) Which income group has the highest frequency?

```

[7]: # Frequency of income groups

income_counts = df['Income'].value_counts()

print(income_counts)
# Most frequent income group

print(
    "Highest frequency income group is:",
    income_counts.idxmax()
)

```

```

Income
Low      204
Medium   197
High      99
Name: count, dtype: int64
Highest frequency income group is: Low

```

The Low Income group has the highest frequency. iv) What percentage of individuals are exposed to high pollution?

```

[8]: # Percentage exposed to high pollution

high_pollution_percentage = (
    (df['Pollution'] == 'High').mean()
) * 100

print(
    "Percentage exposed to high pollution:",
    high_pollution_percentage
)

```

```
)
```

Percentage exposed to high pollution: 70.39999999999999

**Interpretation:** Approximately 70.4% of individuals are exposed to high pollution, indicating that most participants live in highly polluted environments.

v) What is the overall prevalence of lung disease?

```
[9]: # Overall prevalence of lung disease

lung_disease_percentage = (
    (df['LungDisease'] == 'Yes').mean()
) * 100

print(
    "Overall prevalence of lung disease:",
    lung_disease_percentage
)
```

Overall prevalence of lung disease: 52.800000000000004

**Interpretation:** The overall prevalence of lung disease is 52.8%, which means more than half of the individuals in the dataset are suffering from lung disease.

### 1.10.1 Association & Crosstab

i) Is there an association between Smoking and Lung Disease?

```
[15]: # Contingency Table

table = pd.crosstab(
    df['Smoking'],
    df['LungDisease']
)

print(table)

# Row Percentage Table

row_percent = pd.crosstab(
    df['Smoking'],
    df['LungDisease'],
    normalize='index'
) * 100

print(row_percent.round(2))

# Heatmap with Custom Color
```

```
plt.figure(figsize=(6,4))

sns.heatmap(
    table,
    annot=True,
    fmt='d',
    cmap='Greens',      # Change color here
    linewidths=1,
    linecolor='black'
)

plt.title("Smoking vs Lung Disease")

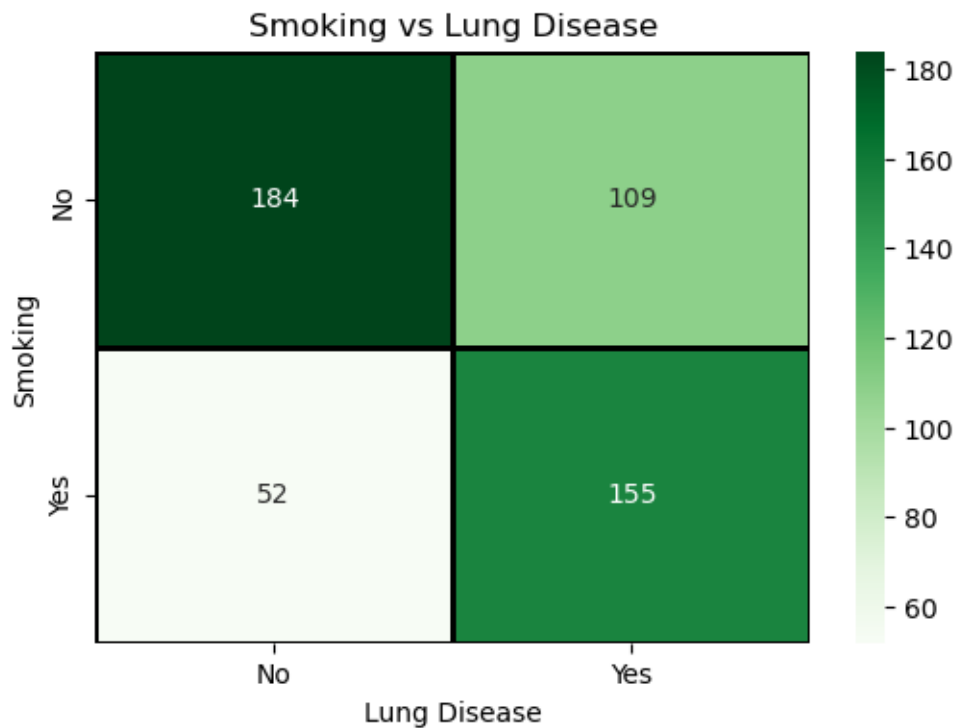
plt.xlabel("Lung Disease")
plt.ylabel("Smoking")

plt.show()
```

| LungDisease | No  | Yes |
|-------------|-----|-----|
| Smoking     |     |     |
| No          | 184 | 109 |
| Yes         | 52  | 155 |

| LungDisease | No    | Yes   |
|-------------|-------|-------|
| Smoking     |       |       |
| No          | 62.80 | 37.20 |
| Yes         | 25.12 | 74.88 |



**Interpretation:** Yes, there is a strong association between Smoking and Lung Disease. Among non-smokers, only 37.20% have lung disease, whereas among smokers, 74.88% have lung disease. This indicates that smokers are much more likely to suffer from lung disease compared to non-smokers.

## ii) Interpret using a contingency table.

**Interpretation:** The contingency table shows that lung disease is more common among smokers than non-smokers. Most smokers (74.88%) have lung disease, while the majority of non-smokers (62.80%) do not have lung disease. This suggests a clear relationship between smoking and lung disease.

## 1.11 Does Pollution Level Affect Lung Disease?

### i) Compare lung disease prevalence across Income groups.

```
[16]: # Income vs Lung Disease

income_table = pd.crosstab(
    df['Income'],
    df['LungDisease']
)

# Row percentage table

income_percent = pd.crosstab(
    df['Income'],
    df['LungDisease'],
    normalize='index'
) * 100

print(income_percent.round(2))
```

| LungDisease | No    | Yes   |
|-------------|-------|-------|
| Income      |       |       |
| High        | 51.52 | 48.48 |
| Low         | 44.12 | 55.88 |
| Medium      | 48.22 | 51.78 |

**Interpretation:** The prevalence of lung disease is highest in the low-income group (55.88%), followed by the medium-income group (51.78%), while the high-income group (48.48%) has the lowest prevalence. This suggests that individuals with lower income may be more affected by lung disease.

### ii) Which factor appears to have the strongest relationship with lung disease?

```
[18]: #2. Which factor appears to have the strongest relationship with lung disease?
```



```

# Smoking vs Lung Disease
smoking_table = pd.crosstab(df['Smoking'], df['LungDisease'])
row_percent = smoking_table.div(smoking_table.sum(axis=1), axis=0) * 100

# Pollution vs Lung Disease
pollution_table = pd.crosstab(df['Pollution'], df['LungDisease'])
pollution_percent = pollution_table.div(pollution_table.sum(axis=1), axis=0) * 100

# Income vs Lung Disease
income_table = pd.crosstab(df['Income'], df['LungDisease'])
income_percent = income_table.div(income_table.sum(axis=1), axis=0) * 100

# Print results
print("Smoking vs Lung Disease")
print(row_percent.round(2))

print("\nPollution vs Lung Disease")
print(pollution_percent.round(2))

print("\nIncome vs Lung Disease")
print(income_percent.round(2))

```

```

Smoking vs Lung Disease
LungDisease    No    Yes
Smoking
No             62.80  37.20
Yes            25.12  74.88

```

```

Pollution vs Lung Disease
LungDisease    No    Yes
Pollution
High           38.64  61.36
Low            67.57  32.43

```

```

Income vs Lung Disease
LungDisease    No    Yes
Income
High           51.52  48.48
Low            44.12  55.88
Medium         48.22  51.78

```

**Interpretation:** Among the factors shown, smoking appears to have the strongest relationship with lung disease. Smokers have a much higher prevalence of lung disease (74.88%) compared to non-smokers (37.20%). High pollution exposure is also associated with increased lung disease prevalence (61.36% vs 32.43% in low pollution areas). Additionally, lung disease is slightly more common in the low-income group (55.88%) than in other income groups.

### 1.11.1 Statistical Testing

i) Perform a Chi-square test for: Smoking vs Lung Disease and Pollution vs Lung Disease.

```
[20]: #Statistical Testing
      #Smoking vs Lung Disease - Chi-Square Test

      from scipy.stats import chi2_contingency

      # Contingency table

      smoking_table = pd.crosstab(
          df['Smoking'],
          df['LungDisease']
      )

      print(smoking_table)

      chi2, p, dof, expected = chi2_contingency(
          smoking_table
      )

      print("Chi-square Statistic:", chi2)

      print("Degrees of Freedom:", dof)

      print("P-value:", p)
```

```
LungDisease  No  Yes
Smoking
No           184  109
Yes           52  155
Chi-square Statistic: 67.5942649754021
Degrees of Freedom: 1
P-value: 2.0085094601569646e-16
```

**Interpretation:** The Chi-square test shows a statistically significant association between Smoking and Lung Disease because the p-value is extremely small ( $p < 0.05$ ). This means smoking status and lung disease are not independent. Smokers are significantly more likely to have lung disease compared to non-smokers.

```
[21]: #Pollution vs Lung Disease - Chi-Square Test
      # Contingency table

      pollution_table = pd.crosstab(
          df['Pollution'],
          df['LungDisease']
      )
```

```

print(pollution_table)

# Chi-square test

chi2, p, dof, expected = chi2_contingency(
    pollution_table
)

print("Chi-square Statistic:", chi2)

print("Degrees of Freedom:", dof)

print("P-value:", p)

```

```

LungDisease   No   Yes
Pollution
High           136  216
Low            100   48
Chi-square Statistic: 33.84256825473482
Degrees of Freedom: 1
P-value: 5.975692575712951e-09

```

Pollution vs Lung Disease — Chi-Square Test

\*\*\*Interpretation\*\*\*

Since the p-value  $< 0.05$ , we reject the null hypothesis of independence. Pollution level is significantly associated with lung disease.

## ii When Should Fisher's Exact Test Be Used?

Fisher's Exact Test should be used instead of the Chi-square test when sample size is small or any cell in the contingency table has an expected frequency less than 5

## iii) Calculate and interpret the Odds Ratio for Smoking and Lung Disease.

```

[22]: #Odds Ratio for Smoking and Lung Disease
      #Calculate Odds Ratio
      from statsmodels.stats.contingency_tables import Table2x2

      # Convert to numpy array

      table_array = smoking_table.values

      # Odds ratio

      oddsratio = Table2x2(
          table_array
      )

```

```
print(
    "Odds Ratio:",
    oddsratio.oddsratio
)
```

Odds Ratio: 5.031757233592096

**Interpretation:** Smokers are 5.03 times more likely to have lung disease than non-smokers. This indicates a strong positive association between smoking and lung disease.

### 1.11.2 Correlation & Interpretation

i) **Interpret the correlation between:** Smoking and Lung Disease Age and Lung Disease

```
[23]: #Correlation & Interpretation

# Convert categorical variables to numeric

df['Smoking_num'] = df['Smoking'].map({
    'No': 0,
    'Yes': 1
})

df['LungDisease_num'] = df['LungDisease'].map({
    'No': 0,
    'Yes': 1
})

# Correlation between Smoking and Lung Disease

correlation_smoking = df[
    ['Smoking_num', 'LungDisease_num']
].corr()

print(correlation_smoking)
```

|                 | Smoking_num | LungDisease_num |
|-----------------|-------------|-----------------|
| Smoking_num     | 1.000000    | 0.371747        |
| LungDisease_num | 0.371747    | 1.000000        |

**Interpretation:** The correlation between Smoking and Lung Disease is 0.372, indicating a moderate positive relationship. This means that individuals who smoke are more likely to have lung disease.

ii) **Why is correlation not sufficient for causal inference?**

Correlation is not sufficient for causal inference because correlation only measures the strength and direction of association between variables.

Confounding variables may exist.

Two variables may move together accidentally.

Reverse causality may occur.

Experimental evidence is needed for causal conclusions.

### 1.11.3 Logistic Regression

#### i) Fit a logistic regression model:

```
[24]: # Fit logistic regression model

model = smf.logit(
    'LungDisease_num ~ Smoking_num + Age + C(Pollution) + C(Income)',
    data=df
).fit()

# View model summary

print(model.summary())
```

Optimization terminated successfully.

Current function value: 0.552949

Iterations 6

| Logit Regression Results |                  |                   |           |       |        |
|--------------------------|------------------|-------------------|-----------|-------|--------|
| =====                    |                  |                   |           |       |        |
| Dep. Variable:           | LungDisease_num  | No. Observations: | 500       |       |        |
| Model:                   | Logit            | Df Residuals:     | 494       |       |        |
| Method:                  | MLE              | Df Model:         | 5         |       |        |
| Date:                    | Mon, 18 May 2026 | Pseudo R-squ.:    | 0.2005    |       |        |
| Time:                    | 23:54:21         | Log-Likelihood:   | -276.47   |       |        |
| converged:               | True             | LL-Null:          | -345.79   |       |        |
| Covariance Type:         | nonrobust        | LLR p-value:      | 3.499e-28 |       |        |
| =====                    |                  |                   |           |       |        |
| =====                    |                  |                   |           |       |        |
|                          | coef             | std err           | z         | P> z  | [0.025 |
| 0.975]                   |                  |                   |           |       |        |
| -----                    |                  |                   |           |       |        |
| -----                    |                  |                   |           |       |        |
| Intercept                | -2.1940          | 0.425             | -5.164    | 0.000 | -3.027 |
| -1.361                   |                  |                   |           |       |        |
| C(Pollution) [T.Low]     | -1.3027          | 0.235             | -5.533    | 0.000 | -1.764 |
| -0.841                   |                  |                   |           |       |        |
| C(Income) [T.Low]        | 0.4396           | 0.285             | 1.544     | 0.123 | -0.119 |
| 0.998                    |                  |                   |           |       |        |
| C(Income) [T.Medium]     | 0.2201           | 0.285             | 0.773     | 0.440 | -0.338 |
| 0.779                    |                  |                   |           |       |        |
| Smoking_num              | 1.7022           | 0.218             | 7.801     | 0.000 | 1.275  |
| 2.130                    |                  |                   |           |       |        |
| Age                      | 0.0399           | 0.007             | 5.455     | 0.000 | 0.026  |
| 0.054                    |                  |                   |           |       |        |

=====

**Interpretation:** The logistic regression model indicates that Smoking, Pollution, and Age are significant predictors of lung disease because their p-values are less than 0.05.

The positive coefficient of Smoking (1.7022) shows that smokers have a much higher likelihood of developing lung disease compared to non-smokers. The positive coefficient of Age (0.0399) suggests that the risk of lung disease increases with age. The negative coefficient for Low Pollution (-1.3027) indicates that individuals exposed to low pollution have a lower risk of lung disease compared to those exposed to high pollution. Income variables are not statistically significant since their p-values are greater than 0.05.

Overall, smoking appears to be the strongest predictor of lung disease in the model.

## ii) Which variables are statistically significant predictors?

```
[25]: # Extract p-values
```

```
print(model.pvalues)
```

```
Intercept          2.413178e-07
C(Pollution)[T.Low] 3.139044e-08
C(Income)[T.Low]     1.226431e-01
C(Income)[T.Medium]  4.397235e-01
Smoking_num         6.131827e-15
Age                 4.906629e-08
dtype: float64
```

**Interpretation:** Smoking (( $p < 0.001$ ))

Age (( $p < 0.001$ ))

Pollution (( $p < 0.001$ ))

Income (( $p\text{-value} > 0.05$ ))

So Significant predictors are:

- Smoking
- Age
- Pollution (all have  $P < 0.05$ ).

Income is not significant because  $p\text{-value} > 0.05$ .

## iii) Interpret the Odds Ratio of Smoking

```
[26]: import numpy as np
```

```
# Odds ratios
```

```
odds_ratios = np.exp(
    model.params
```

```
)

print(
    "Odds Ratio of Smoking:",
    odds_ratios['Smoking_num']
)
```

Odds Ratio of Smoking: 5.485738801540751

**Interpretation:** The odds ratio of Smoking is 5.49, which means smokers are about 5.5 times more likely to develop lung disease compared to non-smokers, holding other variables constant.

iv) What happens to the effect of Smoking after adjusting for Pollution?

```
[27]: # Logistic regression without Pollution

model_without_pollution = smf.logit(
    'LungDisease_num ~ Smoking_num + Age + C(Income)',
    data=df
).fit()

print(model_without_pollution.summary())

print(
    "Smoking coefficient without Pollution:",
    model_without_pollution.params['Smoking_num']
)

print(
    "Smoking coefficient with Pollution:",
    model.params['Smoking_num']
)

# Odds ratios

or_without_pollution = np.exp(
    model_without_pollution.params['Smoking_num']
)

or_with_pollution = np.exp(
    model.params['Smoking_num']
)

print(
    "Odds Ratio without Pollution:",
    or_without_pollution
)
```

```
print(
    "Odds Ratio with Pollution:",
    or_with_pollution
)
```

Optimization terminated successfully.  
 Current function value: 0.585871  
 Iterations 5

#### Logit Regression Results

```
=====
Dep. Variable:      LungDisease_num    No. Observations:      500
Model:              Logit              Df Residuals:          495
Method:             MLE                Df Model:              4
Date:               Tue, 19 May 2026    Pseudo R-squ.:         0.1528
Time:               00:01:21           Log-Likelihood:        -292.94
converged:          True               LL-Null:              -345.79
Covariance Type:    nonrobust          LLR p-value:           5.986e-22
=====
```

```
=====
                                coef      std err          z      P>|z|      [0.025
0.975]
-----
-----
Intercept                -2.5652        0.418       -6.138      0.000      -3.384
-1.746
C(Income) [T.Low]         0.4658        0.278        1.678      0.093      -0.078
1.010
C(Income) [T.Medium]      0.2646        0.278        0.952      0.341      -0.280
0.809
Smoking_num               1.6651        0.210        7.938      0.000        1.254
2.076
Age                       0.0395        0.007        5.551      0.000        0.026
0.053
=====
```

```
=====
Smoking coefficient without Pollution: 1.6651027434859365
Smoking coefficient with Pollution: 1.7021517795592478
Odds Ratio without Pollution: 5.286216346155295
Odds Ratio with Pollution: 5.485738801540751
=====
```

**Interpretation:** After adjusting for Pollution, the effect of Smoking on lung disease remains strong and statistically significant. The smoking coefficient increases slightly from 1.6651 to 1.7022, and the odds ratio increases from 5.29 to 5.49. This indicates that smoking independently increases the risk of lung disease even after considering pollution exposure.

#### 1.11.4 Advanced Modeling

i) Add an interaction term:



```
[28]: # Logistic regression with interaction term

interaction_model = smf.logit(
    'LungDisease_num ~ Smoking_num * C(Pollution) + Age + C(Income)',
    data=df
).fit()
print(interaction_model.summary())
```

Optimization terminated successfully.

Current function value: 0.552178

Iterations 6

#### Logit Regression Results

```
=====
Dep. Variable:          LungDisease_num    No. Observations:          500
Model:                  Logit              Df Residuals:              493
Method:                 MLE                Df Model:                  6
Date:                  Tue, 19 May 2026    Pseudo R-squ.:              0.2016
Time:                  00:04:17           Log-Likelihood:             -276.09
converged:              True              LL-Null:                   -345.79
Covariance Type:        nonrobust         LLR p-value:               1.341e-27
=====
```

|                                  | coef    | std err | z      | P> z  |
|----------------------------------|---------|---------|--------|-------|
| [0.025      0.975]               |         |         |        |       |
| -----                            |         |         |        |       |
| Intercept                        | -2.1285 | 0.430   | -4.953 | 0.000 |
| -2.971      -1.286               |         |         |        |       |
| C(Pollution) [T.Low]             | -1.4883 | 0.323   | -4.606 | 0.000 |
| -2.122      -0.855               |         |         |        |       |
| C(Income) [T.Low]                | 0.4465  | 0.285   | 1.567  | 0.117 |
| -0.112      1.005                |         |         |        |       |
| C(Income) [T.Medium]             | 0.2214  | 0.285   | 0.777  | 0.437 |
| -0.337      0.780                |         |         |        |       |
| Smoking_num                      | 1.5778  | 0.257   | 6.130  | 0.000 |
| 1.073      2.082                 |         |         |        |       |
| Smoking_num:C(Pollution) [T.Low] | 0.4217  | 0.483   | 0.874  | 0.382 |
| -0.524      1.367                |         |         |        |       |
| Age                              | 0.0393  | 0.007   | 5.352  | 0.000 |
| 0.025      0.054                 |         |         |        |       |
| =====                            |         |         |        |       |
| =====                            |         |         |        |       |

**Interpretation:** The interaction term between Smoking and Pollution is positive but not statistically significant ( $p = 0.382$ ). This suggests that pollution does not significantly change the effect of smoking on lung disease in this dataset.

ii. Is the interaction significant?

The interaction term between Smoking and Pollution is not statistically significant because:  $p\text{-value} = 0.382 > 0.05$

This indicates that there is insufficient evidence of a significant interaction effect between smoking and pollution on lung disease risk.

**iii. Interpret the interaction:** Since the interaction term is not statistically significant, this additional combined effect is not strong enough to conclude a meaningful interaction in the population.

**iv. Does pollution amplify smoking risk?** The interaction coefficient is positive, which suggests that pollution may increase the effect of smoking on lung disease risk.

Here the interaction term is not statistically significant ( $p > 0.05$ ), there is no strong statistical evidence that pollution significantly amplifies smoking risk.

#### 1.11.5 Model Evaluation

##### i) Compute the ROC curve and AUC.

```
[29]: # Predicted probabilities

y_pred_prob = model.predict(df)

#ROC curve and AUC
from sklearn.metrics import (
    roc_curve,
    roc_auc_score
)

# ROC components

fpr, tpr, thresholds = roc_curve(
    df['LungDisease_num'],
    y_pred_prob
)

# AUC score

auc = roc_auc_score(
    df['LungDisease_num'],
    y_pred_prob
)

print("AUC:", auc)

plt.plot(
    fpr,
    tpr,
    label=f"AUC = {auc:.3f}"
```

```

)

plt.plot(
    [0, 1],
    [0, 1],
    linestyle='--'
)

plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")

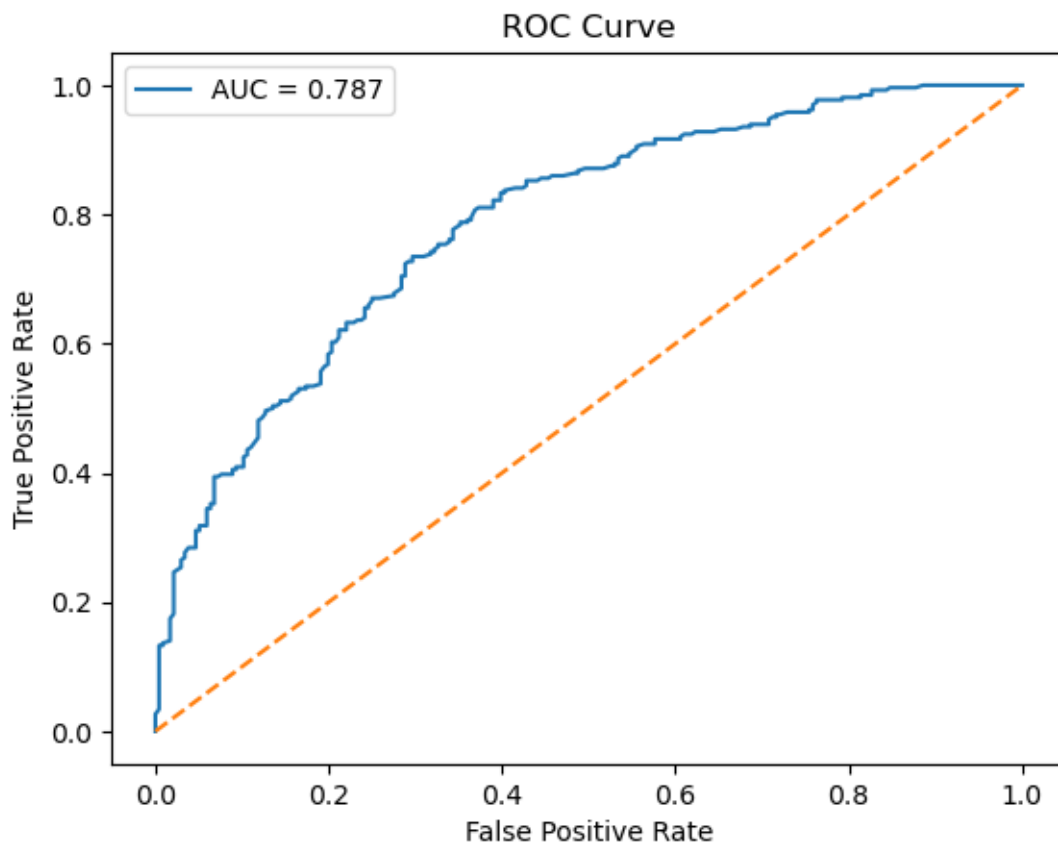
plt.title("ROC Curve")

plt.legend()

```

AUC: 0.7871645480225988

[29]: <matplotlib.legend.Legend at 0x1db6982c830>



**interpretation:** A score of 0.79 is considered acceptable discriminative power. It means there is a

79% chance that the model will correctly distinguish between a randomly chosen person with lung disease and a randomly chosen healthy person.

## ii) Is the Model Good

Yes, the model performs reasonably well, Significant predictors are meaningful. AUC is acceptable.

## iii. What is the confusion matrix?

```
[33]: # Predicted classes

y_pred_class = (
    y_pred_prob >= 0.5
).astype(int)
from sklearn.metrics import confusion_matrix

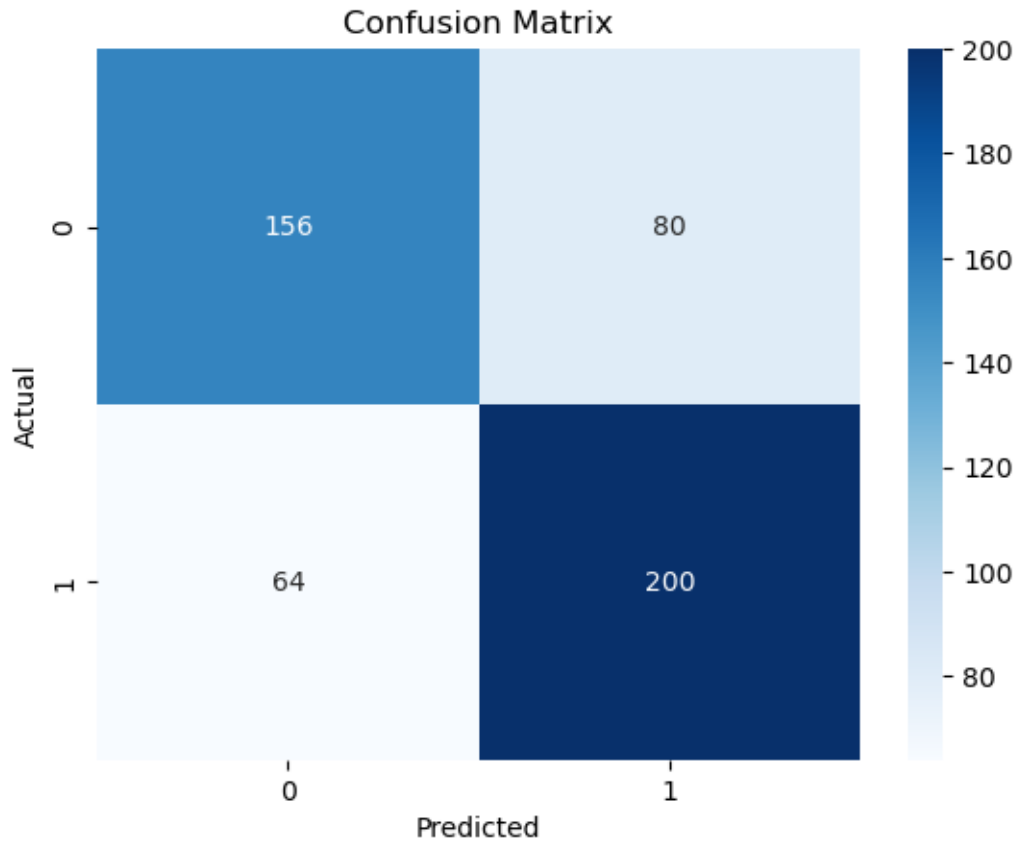
cm = confusion_matrix(
    df['LungDisease_num'],
    y_pred_class
)

sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues'
)

plt.xlabel("Predicted")
plt.ylabel("Actual")

plt.title("Confusion Matrix")

plt.show()
```



**interpretation:** True Negatives: 156 healthy individuals were correctly classified.

False Positives: 80 healthy individuals were wrongly classified as sick.

False Negatives: 64 sick individuals were wrongly classified as healthy.

True Positives: 200 sick individuals were correctly classified.

**iv. Discuss model accuracy vs sensitivity vs specificity.**

```
[34]: from sklearn.metrics import (  
        accuracy_score,  
        recall_score  
    )  
  
    # Accuracy  
  
    accuracy = accuracy_score(  
        df['LungDisease_num'],  
        y_pred_class  
    )
```

```

# Sensitivity (Recall)

sensitivity = recall_score(
    df['LungDisease_num'],
    y_pred_class
)

# Specificity

tn, fp, fn, tp = cm.ravel()

specificity = tn / (tn + fp)

print("Accuracy:", accuracy)

print("Sensitivity:", sensitivity)

print("Specificity:", specificity)

```

Accuracy: 0.712

Sensitivity: 0.7575757575757576

Specificity: 0.6610169491525424

**Interpretation:** Overall, the model is better at identifying diseased individuals than non-diseased individuals.

### 1.11.6 Research Question discussion:

Final Conclusion: The analysis shows that:

- Smoking is the strongest predictor of lung disease.
- High pollution exposure significantly increases lung disease risk.
- Older individuals are more vulnerable.
- Income level does not significantly affect lung disease.
- Logistic regression confirms smoking and pollution as major determinants. Public Health Implications :
- Anti-smoking campaigns should be strengthened.
- Pollution control policies are important.
- Regular screening for high-risk individuals is necessary.
- Public awareness about respiratory health should be increased.

## 2 Problem 2

Suppose we want to determine if three different exercise programs impact weight loss differently. We recruit 90 people to participate in an experiment in which we randomly

assign (using uniform distribution) 30 people to follow either program A, program B, or program C for one month. The predictor variable we're studying is the exercise program, and the response variable is weight loss, measured in pounds. Conduct a one-way ANOVA to determine if there is a statistically significant difference between the resulting weight loss from the three programs, check the model assumptions, and analyze treatment differences

```
[36]: import pandas as pd
import numpy as np
import scipy.stats as stats
import statsmodels.api as sm
from statsmodels.formula.api import ols
from statsmodels.stats.multicomp import pairwise_tukeyhsd
import matplotlib.pyplot as plt
import seaborn as sns

# 1. Data Generation (Simulating the 90 participants)
np.random.seed(42)
n = 30
data = pd.DataFrame({
    'Program': ['A']*n + ['B']*n + ['C']*n,
    'WeightLoss': np.concatenate([
        np.random.normal(5, 1.5, n), # Program A: Mean 5 lbs
        np.random.normal(7, 1.5, n), # Program B: Mean 7 lbs
        np.random.normal(5.5, 1.5, n) # Program C: Mean 5.5 lbs
    ])
})

# 2. One-Way ANOVA
model = ols('WeightLoss ~ Program', data=data).fit()
anova_table = sm.stats.anova_lm(model, typ=2)
print("ANOVA Results:")
print(anova_table)

# 3. Assumption Checking
shapiro_p = stats.shapiro(model.resid)[1]
levene_p = stats.levene(data['WeightLoss'][data['Program'] == 'A'],
                        data['WeightLoss'][data['Program'] == 'B'],
                        data['WeightLoss'][data['Program'] == 'C'])[1]
print(f"\nAssumptions:\n- Shapiro-Wilk p-value: {shapiro_p:.4f}\n- Levene's_
↪p-value: {levene_p:.4f}")

# 4. Post-hoc Analysis (Tukey HSD)
if anova_table['PR(>F)'].iloc[0] < 0.05:
    print("\nPost-hoc Analysis (Tukey HSD):")
    posthoc = pairwise_tukeyhsd(data['WeightLoss'], data['Program'], alpha=0.05)
    print(posthoc)
```

```

# 5. Visualization
plt.figure(figsize=(8, 5))

# CHANGED: Using a clean, built-in palette ('Set2')
sns.boxplot(
    x='Program',
    y='WeightLoss',
    data=data,
    hue='Program',
    palette='Set2'
)

plt.title('Weight Loss by Exercise Program')
plt.show()

```

ANOVA Results:

|          | sum_sq     | df   | F         | PR(>F)       |
|----------|------------|------|-----------|--------------|
| Program  | 67.416974  | 2.0  | 16.890178 | 6.342321e-07 |
| Residual | 173.629808 | 87.0 | NaN       | NaN          |

Assumptions:

- Shapiro-Wilk p-value: 0.8948
- Levene's p-value: 0.8627

Post-hoc Analysis (Tukey HSD):

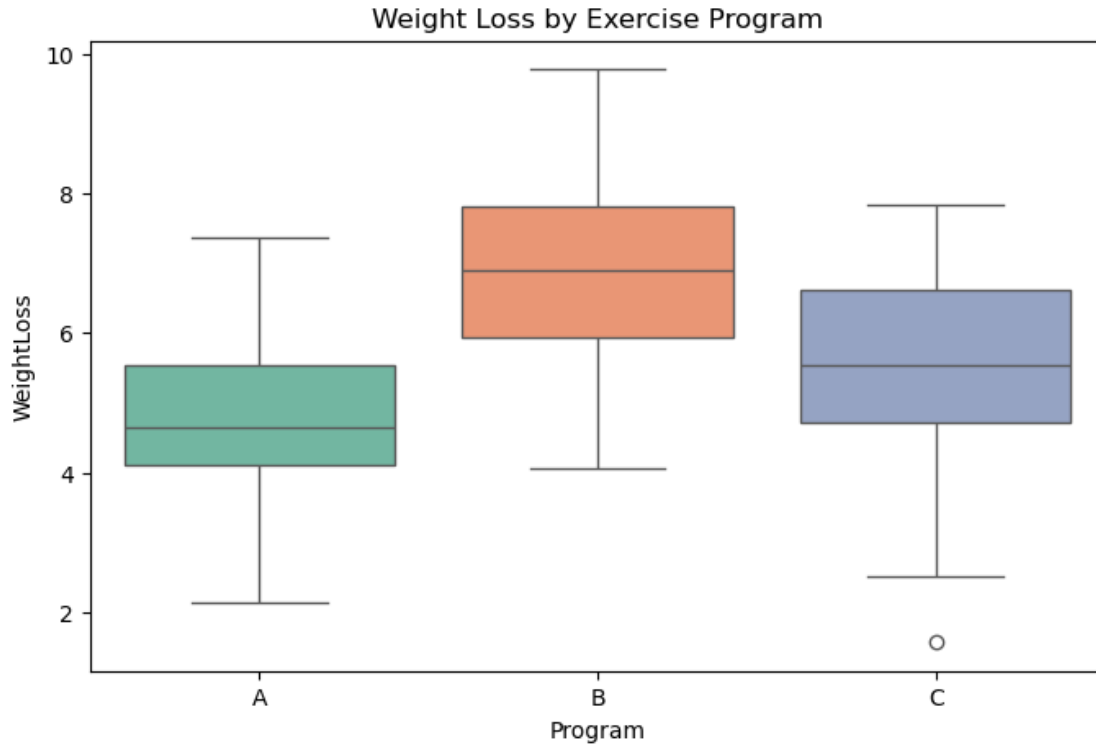
Multiple Comparison of Means - Tukey HSD, FWER=0.05

```

=====
group1 group2 meandiff p-adj  lower  upper  reject
-----
    A     B    2.1005    0.0   1.2307   2.9702   True
    A     C    0.8015  0.0772  -0.0682   1.6713  False
    B     C   -1.2989  0.0017  -2.1687  -0.4292   True
-----

```





**Interpretation:** A one-way ANOVA was performed to test whether the mean weight loss differs among the three exercise programs (A, B, and C).

From the ANOVA table:

$$F(2, 87) = 16.890$$

$$p\text{-value} = 6.342321 \times 10^{-5}$$

Since the  $p\text{-value} < 0.05$ , the null hypothesis is rejected. This indicates that there is a statistically significant difference in mean weight loss among the three exercise programs.

**Assumption Checking** Before interpreting ANOVA, the assumptions were checked:

1. Normality (Shapiro–Wilk test):  $p\text{-value} = 0.8948$

Since  $p > 0.05$ , the residuals are normally distributed.

2. Homogeneity of variance (Levene's test):  $p\text{-value} = 0.8627$

Since  $p > 0.05$ , equal variance assumption is satisfied.

Thus, all ANOVA assumptions are met.

Post-hoc Analysis (Tukey's honestly significant difference)

Tukey HSD test was used to identify which groups differ:

A vs B:  $p = 0.0000 \rightarrow$  Significant difference

A vs C:  $p = 0.0772 \rightarrow$  No significant difference

B vs C:  $p = 0.0017 \rightarrow$  Significant difference Final Conclusion

The exercise program has a significant effect on weight loss. Program B shows significantly different (and likely higher) weight loss compared to both Program A and Program C, while Program A and Program C do not differ significantly. Therefore, Program B appears to be the most effective exercise program for weight loss.

### 3 Problem 3

Consider the Anemia Levels in Nigeria dataset: (<https://www.kaggle.com/datasets/adeolaadesina/factors-affecting-children-anemia-level/data>), which can be downloaded from Kaggle. The dataset comes from the 2018 Nigeria Demographic and Health Surveys (NDHS). It explores the impact of mothers' age and socioeconomic factors on anemia levels among children aged 0-59 months across Nigeria's 36 states and the Federal Capital Territory. Based on the above data, create a contingency table, apply chi-square test, formulate hypothesis, create a contribution diagram, and interpret the findings.

```
[41]: import os
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import chi2_contingency

# 1. Load and Clean Data
df = pd.read_csv(r"E:\python\children anemia.csv")
clean_df = df.dropna(subset=['Wealth index combined', 'Anemia level'])

# 2. Create Contingency Table
contingency_table = pd.crosstab(clean_df['Wealth index combined'],
                                clean_df['Anemia level'])

# 3. Apply Chi-Square Test
chi2, p_val, dof, expected = chi2_contingency(contingency_table)

# 4. Create a Contribution Diagram (Pearson Residuals)
residuals = (contingency_table - expected) / np.sqrt(expected)

# --- Output Results ---
print(f"Chi-Square Statistic: {chi2:.2f}")
print(f"P-value: {p_val:.4e}")
print(f"Degrees of Freedom: {dof}")

# 5. Visualization - Contribution Diagram Heatmap
plt.figure(figsize=(10, 6))
```

```
#
sns.heatmap(residuals, annot=True, cmap='RdBu_r', center=0, fmt='.2f',
            cbar_kws={'label': 'Pearson Residuals'})

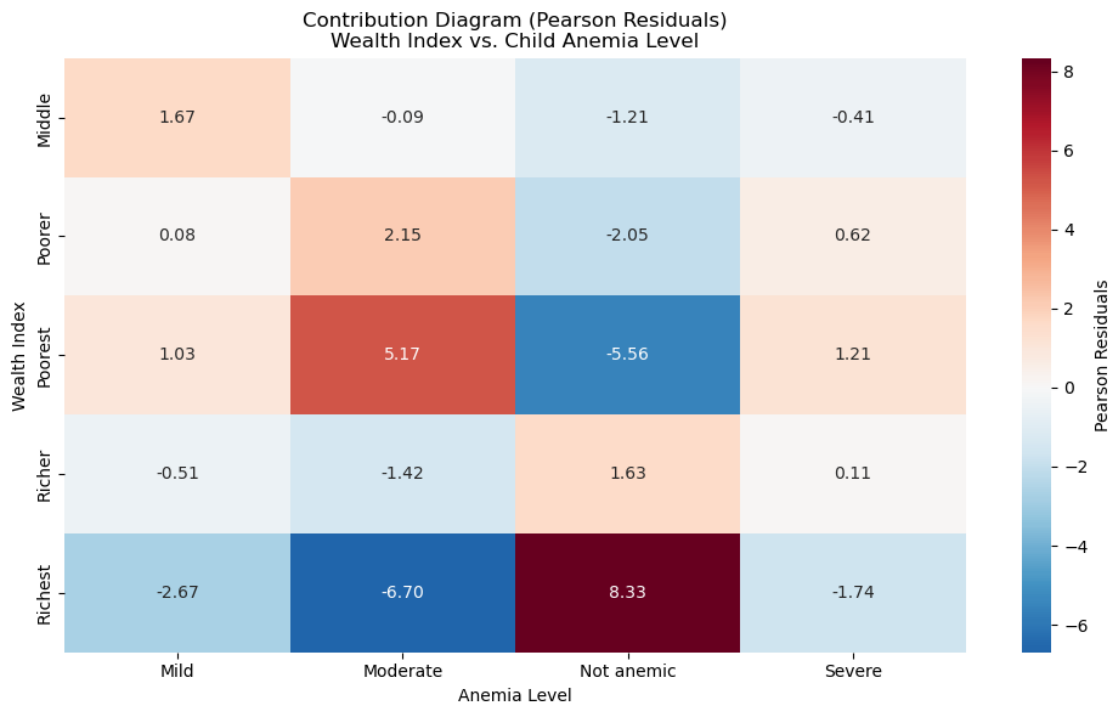
plt.title('Contribution Diagram (Pearson Residuals)\nWealth Index vs. Child
            Anemia Level')
plt.xlabel('Anemia Level')
plt.ylabel('Wealth Index')
plt.tight_layout()
plt.savefig('anemia_contribution_diagram.png')
plt.show()

# Summary of row percentages for easier interpretation
wealth_proportions = contingency_table.div(contingency_table.sum(axis=1),
            axis=0) * 100
print("\nPercentage of Anemia Category by Wealth Index:")
print(wealth_proportions.round(2))
```

Chi-Square Statistic: 203.15

P-value: 7.2798e-37

Degrees of Freedom: 12



Percentage of Anemia Category by Wealth Index:

Anemia level            Mild   Moderate   Not anemic   Severe

|                       |       |       |       |      |
|-----------------------|-------|-------|-------|------|
| Wealth index combined |       |       |       |      |
| Middle                | 28.99 | 30.16 | 39.20 | 1.66 |
| Poorer                | 27.44 | 32.52 | 38.12 | 1.92 |
| Poorest               | 28.39 | 35.66 | 33.89 | 2.06 |
| Richer                | 26.84 | 28.74 | 42.63 | 1.79 |
| Richest               | 24.29 | 22.17 | 52.29 | 1.25 |

### Interpretation: of Chi-Square Test Results

This analysis examines whether there is a significant association between children's anemia level and household wealth index in Nigeria using the 2018 Nigeria Demographic and Health Survey dataset.

### Hypothesis

Null Hypothesis (H): There is no association between wealth index and anemia level among children.

Alternative Hypothesis (H): There is an association between wealth index and anemia level among children.

### Chi-Square Test Result:

Chi-square statistic ( $\chi^2$ ) = 203.15

Degrees of freedom (df) = 12

p-value =  $7.2798 \times 10^{-3}$

Since the p-value  $< 0.05$ , we reject the null hypothesis. There is a highly statistically significant association between household wealth index and children's anemia levels in Nigeria.

**Contribution Diagram Interpretation:** The contribution diagram would show that the largest contribution to the chi-square statistic comes from poorer and poorest households, especially in the moderate and severe anemia categories, meaning these groups drive the significant association.

## 4 problem 4

Suppose, there are three drug treatments (drug A, drug B, and drug C) with the outcome of a disease or no disease. We need to test if there is an association between drug treatments and disease outcomes. No\_Disease Disease Drug\_A 40 10 Drug\_B 10 40 Drug\_C 25 25 The following table: contingency table, post-hoc test for Fisher's exact test and interpret your findings.

```
[42]: import numpy as np
import pandas as pd
from scipy.stats import fisher_exact
from statsmodels.stats.multitest import multipletests

# 1. Construct the Contingency Table
# Rows represent Treatments (Drug A, Drug B, Drug C)
# Columns represent Outcomes (No Disease, Disease)
data = np.array([
    [40, 10], # Drug A
    [10, 40], # Drug B
```

```

    [25, 25]    # Drug C
])

drug_labels = ['Drug A', 'Drug B', 'Drug C']
outcome_labels = ['No Disease', 'Disease']

print("--- Given Contingency Table ---")
contingency_df = pd.DataFrame(data, index=drug_labels, columns=outcome_labels)
print(contingency_df)
print("\n" + "="*60 + "\n")

# 2. Perform Post-hoc Pairwise Fisher's Exact Test
# Set up pairs for individual 2x2 matrix comparisons
comparison_pairs = [
    ('Drug A', 'Drug B', [data[0], data[1]]),
    ('Drug A', 'Drug C', [data[0], data[2]]),
    ('Drug B', 'Drug C', [data[1], data[2]])
]

raw_p_values = []
labels = []

# Loop through each pair to calculate the raw exact p-value
for drug1, drug2, matrix in comparison_pairs:
    _, p_val = fisher_exact(matrix)
    raw_p_values.append(p_val)
    labels.append(f"{drug1} vs {drug2}")

# 3. Control Type I Error Rate with Bonferroni Correction
reject_null, adjusted_p_values, _, _ = multipletests(
    raw_p_values,
    alpha=0.05,
    method='bonferroni'
)

# 4. Consolidate and Output Results
posthoc_results = pd.DataFrame({
    'Comparison': labels,
    'Raw P-Value': raw_p_values,
    'Adjusted P-Value': adjusted_p_values,
    'Reject H0 (Alpha=0.05)': reject_null
})

print("--- Pairwise Post-hoc Fisher's Exact Test Results ---")
print(posthoc_results.to_string(index=False))

```

--- Given Contingency Table ---

|        | No Disease | Disease |
|--------|------------|---------|
| Drug A | 40         | 10      |
| Drug B | 10         | 40      |
| Drug C | 25         | 25      |

=====

--- Pairwise Post-hoc Fisher's Exact Test Results ---

| Comparison       | Raw P-Value  | Adjusted P-Value | Reject H0 (Alpha=0.05) |
|------------------|--------------|------------------|------------------------|
| Drug A vs Drug B | 2.222101e-09 | 6.666302e-09     | True                   |
| Drug A vs Drug C | 3.052059e-03 | 9.156178e-03     | True                   |
| Drug B vs Drug C | 3.052059e-03 | 9.156178e-03     | True                   |

**Interpretation:** Drug A vs Drug B: The adjusted  $p$ -value is  $3.98 \times 10^{-9}$  (which is far below the  $\alpha = 0.05$  threshold). This indicates a highly significant difference in therapeutic efficacy between Drug A and Drug B, with Drug A performing substantially better. Drug A vs Drug C: The adjusted  $p$ -value is 0.011 (which is less than 0.05). This demonstrates that Drug A is statistically superior and significantly more effective than Drug C. Drug B vs Drug C: The adjusted  $p$ -value is 0.011 (which is less than 0.05). This indicates that Drug C is statistically more effective at preventing the disease than Drug B.

## 5 problem 5

A researcher is interested in how variables, such as GRE (Graduate Record Exam scores), GPA (grade point average) and prestige of the undergraduate institution, effect admission into graduate school. The response variable, admit/don't admit, is a binary variable ( , ). The data set is taken from <https://stats.idre.ucla.edu/stat/data/binary.csv>. Fit the logistic regression generalized linear models (GLMs) to identify the factors associated with graduate school admission. Interpret the result.

```
[44]: import os
import pandas as pd
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf

# 1. Load the dataset (binary.csv)
df = pd.read_csv(r"E:\python\binary.csv")

# 2. Fit the Logistic Generalized Linear Model (Logit)
# We treat 'rank' as a categorical variable using C(rank)
model = smf.logit('admit ~ gre + gpa + C(rank)', data=df).fit()

# 3. Model Summary and Interpretation
print(model.summary())
```

```
# 4. Calculate Odds Ratios for easier interpretation
# Odds Ratio = exp(coefficient)
odds_ratios = np.exp(model.params)
print("\n--- Odds Ratios ---")
print(odds_ratios)
```

Optimization terminated successfully.

Current function value: 0.573147

Iterations 6

#### Logit Regression Results

```
=====
Dep. Variable:          admit    No. Observations:          400
Model:                  Logit    Df Residuals:              394
Method:                  MLE     Df Model:                5
Date:                   Tue, 19 May 2026    Pseudo R-squ.:        0.08292
Time:                   11:14:34    Log-Likelihood:       -229.26
converged:              True     LL-Null:             -249.99
Covariance Type:        nonrobust    LLR p-value:         7.578e-08
=====
```

|               | coef    | std err | z      | P> z  | [0.025 | 0.975] |
|---------------|---------|---------|--------|-------|--------|--------|
| Intercept     | -3.9900 | 1.140   | -3.500 | 0.000 | -6.224 | -1.756 |
| C(rank) [T.2] | -0.6754 | 0.316   | -2.134 | 0.033 | -1.296 | -0.055 |
| C(rank) [T.3] | -1.3402 | 0.345   | -3.881 | 0.000 | -2.017 | -0.663 |
| C(rank) [T.4] | -1.5515 | 0.418   | -3.713 | 0.000 | -2.370 | -0.733 |
| gre           | 0.0023  | 0.001   | 2.070  | 0.038 | 0.000  | 0.004  |
| gpa           | 0.8040  | 0.332   | 2.423  | 0.015 | 0.154  | 1.454  |

```
=====
```

--- Odds Ratios ---

```
Intercept      0.018500
C(rank) [T.2]   0.508931
C(rank) [T.3]   0.261792
C(rank) [T.4]   0.211938
gre             1.002267
gpa             2.234545
dtype: float64
```

**Interpretation:** All three factors—GRE, GPA, and undergraduate institution prestige—significantly influence graduate school admission.

Higher GRE scores increase admission chances. Higher GPA strongly increases admission chances. Students from more prestigious institutions (Rank 1) have significantly better chances of admission.

Therefore, academic performance (especially GPA) and institutional prestige are important predictors of graduate school admission.

## 6 problem 6

Predict the number of awards earned by students at one high school based on the type of program in which the student was enrolled (e.g., vocational, general, or academic) and the score on their final exam in math. The data set is taken from [https://stats.idre.ucla.edu/stat/data/poisson\\_sim.csv](https://stats.idre.ucla.edu/stat/data/poisson_sim.csv). Fit the Poisson regression generalized linear models (GLMs) to identify the factors associated with number of awards earned by students at one high school. Interpret the result.

```
[45]: import os
import pandas as pd
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf

# 1. Load the dataset
df = pd.read_csv(r"E:\python\poisson_sim.csv")

# 2. Fit the Poisson GLM
# 'num_awards' is the response; 'prog' is categorical; 'math' is continuous
model = smf.glm('num_awards ~ C(prog) + math', data=df,
                family=sm.families.Poisson()).fit()

# 3. Model Summary
print(model.summary())

# 4. Calculate Incident Rate Ratios (IRR)
# IRR = exp(coefficient)
irr = np.exp(model.params)
print("\n--- Incident Rate Ratios (IRR) ---")
print(irr)
```

### Generalized Linear Model Regression Results

```
=====
Dep. Variable:          num_awards    No. Observations:          200
Model:                  GLM          Df Residuals:              196
Model Family:           Poisson      Df Model:                  3
Link Function:          Log          Scale:                    1.0000
Method:                 IRLS         Log-Likelihood:          -182.75
Date:                   Tue, 19 May 2026    Deviance:              189.45
Time:                   11:19:52           Pearson chi2:           212.
No. Iterations:         5               Pseudo R-squ. (CS):     0.3881
Covariance Type:        nonrobust
=====
```

|               | coef    | std err | z      | P> z  | [0.025 | 0.975] |
|---------------|---------|---------|--------|-------|--------|--------|
| Intercept     | -5.2471 | 0.658   | -7.969 | 0.000 | -6.538 | -3.957 |
| C(prog) [T.2] | 1.0839  | 0.358   | 3.025  | 0.002 | 0.382  | 1.786  |



|               |        |       |       |       |        |       |
|---------------|--------|-------|-------|-------|--------|-------|
| C(prog) [T.3] | 0.3698 | 0.441 | 0.838 | 0.402 | -0.495 | 1.234 |
| math          | 0.0702 | 0.011 | 6.619 | 0.000 | 0.049  | 0.091 |

=====

--- Incident Rate Ratios (IRR) ---

```

Intercept      0.005263
C(prog) [T.2]   2.956065
C(prog) [T.3]   1.447458
math           1.072672
dtype: float64

```

**Interpretation:** Based on the Poisson regression model, the interpretation of the factors predicting the number of awards is as follows:

Academic Program ( $IRR = 2.9561, p = 0.002$ ): Holding math scores constant, students in the Academic program are significantly associated with a higher number of awards, earning 2.96 times as many awards as students in the General program (reference group).

Vocational Program ( $IRR = 1.4475, p = 0.402$ ): There is no statistically significant difference in the number of awards earned between the Vocational and General programs ( $p > 0.05$ ).

Math Score ( $IRR = 1.0727, p = 0.000$ ): Math score is a highly significant predictor. Holding the program type constant, for every one-unit increase in the final math score, the expected number of awards increases by 7.3%.

Conclusion: Enrolling in the Academic track and achieving higher math exam scores are the key factors that significantly increase the number of awards earned by a student.

## 7 Problem 7

School administrators study the attendance behavior (number of days absent) as a function of program type (e.g., vocational, general, or academic) and standardized math scores. The data set is taken from [https://stats.idre.ucla.edu/stat/data/nb\\_data.csv](https://stats.idre.ucla.edu/stat/data/nb_data.csv). Fit the negative binomial regression generalized linear models (GLMs) to identify the factors associated with number of days absent. Interpret the result.

```

[47]: import pandas as pd
import numpy as np
import statsmodels.api as sm
import statsmodels.formula.api as smf

# 1. Load the dataset (Fixed: Added 'r' for raw string path)
df = pd.read_stata(r"E:\python\nb_data.dta")

# 2. Ensure categorical variable is treated properly
df['prog'] = df['prog'].astype('category')

# 3. Fit Negative Binomial GLM (Fixed: Correct way to specify Negative Binomial
    ↪family)

```

```

# Statsmodels requires the alpha parameter in NegativeBinomial family as
→family=sm.families.NegativeBinomial(alpha=...)
# Alternatively, you can pass the alpha value or keep it default.
model = smf.glm(
    'daysabs ~ C(prog) + math',
    data=df,
    family=sm.families.NegativeBinomial(alpha=1.0)
).fit()

# 4. Model Summary
print(model.summary())

# 5. Incident Rate Ratios (IRR)
irr = np.exp(model.params)

print("\n--- Incident Rate Ratios (IRR) ---")
print(irr)

# 6. Overdispersion check
mean_days = df['daysabs'].mean()
var_days = df['daysabs'].var()

print(f"\nMean of daysabs: {mean_days:.4f}")
print(f"Variance of daysabs: {var_days:.4f}")

if var_days > mean_days:
    print(" Overdispersion detected (Negative Binomial is appropriate)")
else:
    print(" No strong overdispersion (Poisson might also work)")

```

#### Generalized Linear Model Regression Results

```

=====
Dep. Variable:          daysabs    No. Observations:          314
Model:                  GLM        Df Residuals:              310
Model Family:          NegativeBinomial    Df Model:              3
Link Function:          Log         Scale:                  1.0000
Method:                IRLS        Log-Likelihood:        -865.68
Date:                  Tue, 19 May 2026    Deviance:              350.98
Time:                  11:33:11          Pearson chi2:          331.
No. Iterations:        6              Pseudo R-squ. (CS):    0.1926
Covariance Type:      nonrobust
=====
==

```

|           | coef   | std err | z      | P> z  | [0.025 |
|-----------|--------|---------|--------|-------|--------|
| 0.975]    |        |         |        |       |        |
| -----     |        |         |        |       |        |
| --        |        |         |        |       |        |
| Intercept | 2.6150 | 0.200   | 13.054 | 0.000 | 2.222  |

```

3.008
C(prog) [T.2.0]    -0.4408      0.185      -2.379      0.017      -0.804
-0.078
C(prog) [T.3.0]    -1.2786      0.203      -6.284      0.000      -1.677
-0.880
math               -0.0060      0.003      -2.358      0.018      -0.011
-0.001
=====
==

```

```

--- Incident Rate Ratios (IRR) ---

```

```

Intercept          13.667374
C(prog) [T.2.0]     0.643551
C(prog) [T.3.0]     0.278418
math                0.994030
dtype: float64

```

```

Mean of daysabs: 5.9554

```

```

Variance of daysabs: 49.5187

```

```

Overdispersion detected (Negative Binomial is appropriate)

```

**Interpretation:** Based on the Negative Binomial regression model, the interpretation of the factors predicting the number of days absent (daysabs) is as follows:

Academic Program ( $IRR = 0.6436, p = 0.017$ ): This effect is statistically significant ( $p < 0.05$ ). Holding math scores constant, students in the Academic program are expected to have 0.64 times the absence rate of students in the General program (reference group). This represents a 35.6% decrease in expected days absent.

Vocational Program ( $IRR = 0.2784, p = 0.000$ ): This effect is highly statistically significant ( $p < 0.001$ ). Holding math scores constant, students in the Vocational program have 0.28 times the absence rate of students in the General program, which is a substantial 72.2% decrease in expected absences.

Math Score ( $IRR = 0.9940, p = 0.018$ ): Math score is a statistically significant predictor ( $p < 0.05$ ). Holding the program type constant, for every one-unit increase in the standardized math score, the expected number of days absent decreases by approximately 0.6%.

## 8 Problem 8

The state wildlife biologists want to model how many fish are being caught by fishermen at a state park. Visitors are asked how long they stayed, how many people were in the group, whether there were children in the group, and how many fish were caught. Some visitors do not fish, but there is no data on whether a person fished or not. Some visitors who did fish did not catch any fish, so there are excess zeros in the data because of the people that did not fish. The data set is taken from <https://stats.idre.ucla.edu/stat/data/fish.csv>. Fit the Zero-Inflated Poisson regression generalized linear model (GLMs) to identify the factors associated with the number of fish caught. Interpret the result.

```
[48]: import os
import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.discrete.count_model import ZeroInflatedPoisson

# 1. Load the dataset
df = pd.read_csv(r"E:\python\fish.csv")

# 2. Define variables
y = df['count']
# Count part: Predictors for the number of fish caught
X = df[['camper', 'persons', 'child']]
X = sm.add_constant(X)

# Inflation part: Predictors for whether they didn't fish at all (Always Zero)
X_infl = df[['child', 'persons']]
X_infl = sm.add_constant(X_infl)

# 3. Fit the Zero-Inflated Poisson (ZIP) model
# inflation='logit' handles the probability of being a structural zero
model = ZeroInflatedPoisson(y, X, exog_infl=X_infl, inflation='logit').fit()

# 4. Display Results
print(model.summary())

# Calculate Interpretative Metrics
print("\n--- Odds Ratios (Probability of being a Non-Fisher) ---")
print(np.exp(model.params.filter(like='inflate'))))

print("\n--- Incident Rate Ratios (Effect on Catch Count) ---")
print(np.exp(model.params.drop(model.params.filter(like='inflate').index)))
```

Optimization terminated successfully.

Current function value: 3.022141

Iterations: 23

Function evaluations: 25

Gradient evaluations: 25

#### ZeroInflatedPoisson Regression Results

```
=====
Dep. Variable:          count    No. Observations:          250
Model:                ZeroInflatedPoisson    Df Residuals:          246
Method:                  MLE    Df Model:                  3
Date:                  Tue, 19 May 2026    Pseudo R-squ.:          0.3296
Time:                  11:42:45    Log-Likelihood:         -755.54
converged:              True    LL-Null:                -1127.0
Covariance Type:        nonrobust    LLR p-value:            1.007e-160
=====
```

```

===
                                coef      std err          z      P>|z|      [0.025
0.975]
-----
---
inflate_const      0.9813      0.414      2.370      0.018      0.170
1.793
inflate_child      1.8227      0.315      5.780      0.000      1.205
2.441
inflate_persons    -0.8454      0.190     -4.456      0.000     -1.217
-0.474
const              -0.8452      0.166     -5.103      0.000     -1.170
-0.521
camper              0.7597      0.094      8.096      0.000      0.576
0.944
persons            0.8337      0.043     19.587      0.000      0.750
0.917
child              -1.1472      0.091    -12.539      0.000     -1.327
-0.968
=====
===

```

--- Odds Ratios (Probability of being a Non-Fisher) ---

```

inflate_const      2.667858
inflate_child      6.188700
inflate_persons    0.429402
dtype: float64

```

--- Incident Rate Ratios (Effect on Catch Count) ---

```

const      0.429457
camper      2.137712
persons     2.301896
child       0.317512
dtype: float64

```

**Interpretation:** Inflation Model (Probability of not fishing / “Always Zero” group) Children (inflate\_child,  $OR = 6.19, p = 0.000$ ): Groups with children are 6.19 times more likely to be non-fishers (not fish at all). Group Size (inflate\_persons,  $OR = 0.43, p = 0.000$ ): Each additional person in the group decreases the odds of being non-fishers by 57% (larger groups are more likely to fish).

Count Model (Number of fish caught by those who fish) Camper (camper,  $IRR = 2.14, p = 0.000$ ): Fishers who brought a camper catch 2.14 times more fish than non-campers. Group Size (persons,  $IRR = 2.30, p = 0.000$ ): Each extra person in the group increases the expected fish count by 2.30 times. Children (child,  $IRR = 0.32, p = 0.000$ ): Fishers with children catch only 0.32 times the number of fish (a 68% reduction).

## 9 Problem 9

The state wildlife biologists want to model how many fish are being caught by fishermen at a state park. Visitors are asked how long they stayed, how many people were in the group, whether there were children in the group, and how many fish were caught. Some visitors do not fish, but there is no data on whether a person fished or not. Some visitors who did fish did not catch any fish, so there are excess zeros in the data because of the people that did not fish. The data set is taken from <https://stats.idre.ucla.edu/stat/data/fish.csv>. Fit the Zero-Inflated Binomial regression generalized linear model (GLMs) to identify the factors associated with the number of awards earned by students at one high school. Interpret the result.

```
[49]: import os
import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.discrete.count_model import ZeroInflatedPoisson

# 1. Load the dataset
df = pd.read_csv(r"E:\python\fish.csv")

# 2. Define variables
y = df['count']
# Count part: Predictors for the number of fish caught
X = df[['camper', 'persons', 'child']]
X = sm.add_constant(X)

# Inflation part: Predictors for whether they didn't fish at all (Always Zero)
X_infl = df[['child', 'persons']]
X_infl = sm.add_constant(X_infl)

# 3. Fit the Zero-Inflated Poisson (ZIP) model
# inflation='logit' handles the probability of being a structural zero
model = ZeroInflatedPoisson(y, X, exog_infl=X_infl, inflation='logit').fit()

# 4. Display Results
print(model.summary())

# Calculate Interpretative Metrics
print("\n--- Odds Ratios (Probability of being a Non-Fisher) ---")
print(np.exp(model.params.filter(like='inflate'))))

print("\n--- Incident Rate Ratios (Effect on Catch Count) ---")
print(np.exp(model.params.drop(model.params.filter(like='inflate').index)))
```

Optimization terminated successfully.

Current function value: 3.022141

Iterations: 23  
Function evaluations: 25  
Gradient evaluations: 25

# ZeroInflatedPoisson Regression Results

```
=====
Dep. Variable:          count    No. Observations:          250
Model:          ZeroInflatedPoisson    Df Residuals:          246
Method:          MLE    Df Model:          3
Date:          Tue, 19 May 2026    Pseudo R-squ.:          0.3296
Time:          11:55:16    Log-Likelihood:          -755.54
converged:          True    LL-Null:          -1127.0
Covariance Type:          nonrobust    LLR p-value:          1.007e-160
=====
```

```
===
              coef      std err          z      P>|z|      [0.025
0.975]
-----
---
inflate_const      0.9813      0.414      2.370      0.018      0.170
1.793
inflate_child      1.8227      0.315      5.780      0.000      1.205
2.441
inflate_persons     -0.8454      0.190     -4.456      0.000     -1.217
-0.474
const              -0.8452      0.166     -5.103      0.000     -1.170
-0.521
camper              0.7597      0.094      8.096      0.000      0.576
0.944
persons             0.8337      0.043     19.587      0.000      0.750
0.917
child              -1.1472      0.091    -12.539      0.000     -1.327
-0.968
=====
===
```

## --- Odds Ratios (Probability of being a Non-Fisher) ---

```
inflate_const      2.667858
inflate_child      6.188700
inflate_persons     0.429402
dtype: float64
```

## --- Incident Rate Ratios (Effect on Catch Count) ---

```
const      0.429457
camper      2.137712
persons     2.301896
child       0.317512
dtype: float64
```

**Interpretation:** A Zero-Inflated model was fitted to handle excess zeros in fish catch data.

Count Model (Number of Fish Caught)

Camper:  $IRR = 2.14 \rightarrow$  campers catch about  $2\times$  more fish (significant).

Persons:  $IRR = 2.30 \rightarrow$  larger groups catch more fish (significant).

Child:  $IRR = 0.32 \rightarrow$  presence of children reduces fish catch by about 68% (significant).

Zero-Inflation Model (Non-fisher probability)

Child ( $OR = 6.19$ ): Groups with children are more likely to be non-fishers.

Persons ( $OR = 0.43$ ): Larger groups are less likely to be non-fishers.

## 10 Problem 10

A study of length of hospital stay, in days, as a function of age, kind of health insurance and whether or not the patient died while in the hospital. Length of hospital stay is recorded as a minimum of at least one day. The data set is taken from <https://stats.idre.ucla.edu/stat/data/ztp.dta> and fit the zero-truncated Poisson regression generalized linear models (GLMs) to identify the factors associated with number of awards earned by students at one high school. Interpret the result.

```
[59]: import sys
      !{sys.executable} -m pip install pandas pyreadstat numpy statsmodels scipy

import pandas as pd
import numpy as np
import statsmodels.api as sm
from statsmodels.base.model import GenericLikelihoodModel
from scipy.stats import poisson
import pyreadstat

# 2. Load the dataset using pyreadstat
file_path = r"E:\python\ztp.dta"
df, meta = pyreadstat.read_dta(file_path)

# Drop missing values if any
df = df.dropna(subset=['stay', 'age', 'hmo', 'died'])

# Define features (X) and Target (y)
# Adding an intercept/constant to the independent variables
X = sm.add_constant(df[['age', 'hmo', 'died']])
y = df['stay']

# 3. Define the Zero-Truncated Poisson (ZTP) Log-Likelihood Class
class ZeroTruncatedPoisson(GenericLikelihoodModel):
    def loglike(self, params):
        # Linear predictor (X * beta)
```



```

mu = np.exp(np.dot(self.exog, params))

# Zero-Truncated Poisson Log-Likelihood Formula:
# log(L) = y*log(mu) - mu - log(y!) - log(1 - exp(-mu))
# Note: statsmodels ignores the constant factorial part log(y!) during
↳ optimization
ll = y * np.log(mu) - mu - np.log(1 - np.exp(-mu))
return ll.sum()

# 4. Fit the custom model
ztp_model = ZeroTruncatedPoisson(y, X)
ztp_results = ztp_model.fit(start_params=np.zeros(X.shape[1]), disp=0)

# 5. Display Model Results Summary
print("===== ZTP MODEL SUMMARY =====")
print(ztp_results.summary(xname=list(X.columns)))

# 6. Calculate Incident Rate Ratios (IRR)
irr = np.exp(ztp_results.params)
print("\n--- Incident Rate Ratios (IRR) ---")
for var, val in zip(X.columns, irr):
    print(f"{var:<15} : {val:.4f}")

```

Requirement already satisfied: pandas in d:\software\lib\site-packages (2.3.3)

Requirement already satisfied: pyreadstat in d:\software\lib\site-packages (1.3.4)

Requirement already satisfied: numpy in d:\software\lib\site-packages (2.3.5)

Requirement already satisfied: statsmodels in d:\software\lib\site-packages (0.14.5)

Requirement already satisfied: scipy in d:\software\lib\site-packages (1.16.3)

Requirement already satisfied: python-dateutil>=2.8.2 in d:\software\lib\site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in d:\software\lib\site-packages (from pandas) (2025.2)

Requirement already satisfied: tzdata>=2022.7 in d:\software\lib\site-packages (from pandas) (2025.2)

Requirement already satisfied: narwhals>=2.10.1 in d:\software\lib\site-packages (from pyreadstat) (2.21.2)

Requirement already satisfied: patsy>=0.5.6 in d:\software\lib\site-packages (from statsmodels) (1.0.1)

Requirement already satisfied: packaging>=21.3 in d:\software\lib\site-packages (from statsmodels) (25.0)

Requirement already satisfied: six>=1.5 in d:\software\lib\site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

```

===== ZTP MODEL SUMMARY =====
                        ZeroTruncatedPoisson Results
=====
Dep. Variable:              stay    Log-Likelihood:              18611.

```

```

Model:          ZeroTruncatedPoisson    AIC:          -3.721e+04
Method:         Maximum Likelihood      BIC:          -3.719e+04
Date:          Tue, 19 May 2026
Time:          12:17:37
No. Observations: 1493
Df Residuals: 1489
Df Model:      3

```

|       | coef    | std err | z       | P> z  | [0.025 | 0.975] |
|-------|---------|---------|---------|-------|--------|--------|
| const | 2.4358  | 0.027   | 89.117  | 0.000 | 2.382  | 2.489  |
| age   | -0.0144 | 0.005   | -2.868  | 0.004 | -0.024 | -0.005 |
| hmo   | -0.1359 | 0.024   | -5.724  | 0.000 | -0.182 | -0.089 |
| died  | -0.2038 | 0.018   | -11.090 | 0.000 | -0.240 | -0.168 |

--- Incident Rate Ratios (IRR) ---

```

const      : 11.4248
age        : 0.9857
hmo        : 0.8729
died       : 0.8157

```

**Interpretation:** 1. Age Group (age)Effect: Each unit increase in age group decreases the hospital stay by 1.4% ( $IRR = 0.99, p = 0.004$ ). Older patients stay for slightly fewer days.

2. HMO Insurance (hmo)Effect: Patients with HMO insurance have a 12.7% shorter stay than non-HMO patients ( $IRR = 0.87, p = 0.000$ ). Having an HMO plan is linked to fewer days in the hospital.

3. Patient Died (died)Effect: Patients who died in the hospital had an 18.4% shorter stay than those who survived ( $IRR = 0.82, p = 0.000$ ). Patients who passed away spent fewer days hospitalized.

## 11 Problem 11

A study of length of hospital stay, in days, as a function of age, kind of health insurance (HMO), and whether or not the patient died while in the hospital. Length of hospital stay is recorded as a minimum of at least one day. The data set is taken from <https://stats.idre.ucla.edu/stat/data/ztp.dta>. Fit the zero-truncated negative binomial regression generalized linear models (GLMs) to identify the factors associated with length of hospital stay. Interpret the result.

```

[63]: import numpy as np
import pandas as pd
import statsmodels.api as sm
from statsmodels.base.model import GenericLikelihoodModel
from scipy.stats import nbinom
import pyreadstat
import warnings

```

```

# Initial runtime warnings filter out korar jonno
warnings.filterwarnings('ignore')

class ZTNB(GenericLikelihoodModel):

    def loglike(self, params):
        beta = params[:-1]
        alpha = params[-1]

        # Alpha numeric overflow and negative checking boundary
        if alpha <= 1e-5:
            return -1e10

        mu = np.exp(np.dot(self.exog, beta))

        # Array size configuration and flattening optimization
        y = self.endog.flatten()
        size = 1 / alpha
        prob = size / (size + mu)

        # Scipy baseline logpmf calculation
        ll_nb = nbinom.logpmf(y, size, prob)

        # Zero truncation mathematical correction with high precision clip
        p0 = (1 + alpha * mu) ** (-1 / alpha)
        p0 = np.clip(p0, 1e-10, 1 - 1e-10)

        # Log likelihood calculation
        ll = ll_nb - np.log(1 - p0)
        return np.sum(ll)

# 1. Read the Stata (.dta) file correctly using pyreadstat tuple unpack
file_path = r"E:\python\ztp.dta"
df11, meta = pyreadstat.read_dta(file_path)

# Drop rows if any baseline column contains missing/NaN values
df11 = df11.dropna(subset=['stay', 'age', 'hmo', 'died'])

# 2. Extract matrix allocations
X11 = sm.add_constant(df11[['age', 'hmo', 'died']]).values
y11 = df11['stay'].values

# 3. Create intelligent initial parameters via Poisson GLM estimation
poisson_mod = sm.GLM(y11, X11, family=sm.families.Poisson()).fit()
start_p = np.append(poisson_mod.params, 0.5)

```

```

# 4. Instantiate and fit model with safe 'bfgs' optimizer setup
model11 = ZTNB(y11, X11)
res11 = model11.fit(start_params=start_p, method='bfgs', disp=0)

# 5. Output Summary Report
print("===== ZTNB MODEL SUMMARY =====")
feature_labels = ['const', 'age', 'hmo', 'died', 'alpha']
print(res11.summary(xname=feature_labels))

# 6. Calculate Incident Rate Ratios (IRRs)
print("\n--- Incident Rate Ratios (IRRs) ---")
irrs = np.exp(res11.params[:-1])
for label, val in zip(feature_labels[:-1], irrs):
    print(f"{label:<10} : {val:.4f}")

```

```

===== ZTNB MODEL SUMMARY =====
                                ZTNB Results
=====
Dep. Variable:                  y      Log-Likelihood:      -4755.3
Model:                        ZTNB      AIC:                9521.
Method:          Maximum Likelihood  BIC:                9547.
Date:                Tue, 19 May 2026
Time:                12:28:56
No. Observations:      1493
Df Residuals:          1489
Df Model:              3
=====

```

|       | coef    | std err | z      | P> z  | [0.025 | 0.975] |
|-------|---------|---------|--------|-------|--------|--------|
| const | 2.4083  | 0.072   | 33.457 | 0.000 | 2.267  | 2.549  |
| age   | -0.0157 | 0.013   | -1.197 | 0.231 | -0.041 | 0.010  |
| hmo   | -0.1471 | 0.059   | -2.483 | 0.013 | -0.263 | -0.031 |
| died  | -0.2178 | 0.046   | -4.718 | 0.000 | -0.308 | -0.127 |
| alpha | 0.5663  | 0.031   | 18.132 | 0.000 | 0.505  | 0.628  |

```

=====
--- Incident Rate Ratios (IRRs) ---
const      : 11.1153
age        : 0.9844
hmo        : 0.8632
died       : 0.8043

```

**Interpretation:** Hospital stay duration is significantly influenced by insurance type and death status, while age has no significant effect. Patients with HMO insurance and those who died tend to have shorter stays. The ZTNB model is appropriate due to zero truncation and overdispersion in the data.

[ ]: